AI Usage Journal

ARI 2129 — Principles of Computer Vision for AI

Topic: Convolution Kernels • Author: James • Role: Presentation Slides

---

Section 1: Tools Declared

The following AI tools were used during the production of the presentation slides deliverable.

Tool	Version	Purpose
Claude (Anthropic)	Claude Sonnet 4.5	Drafting and refining slide content, checking calculations, generating analogies, and improving descriptions of technical concepts.

Section 2: Notable Prompts

The following prompts represent moments that genuinely changed or improved the work, not routine interactions.

Prompt 1

Prompt: *"Check that these computations are correct or incorrect. Example: Input=224, K=3, P=1, S=1 → O = (224-3+0)/1 + 1 = 222"*

Response summary: Claude identified that the second example contained two errors — the padding term was written as 0 instead of 2P=2, and the result 222 was wrong. The correct answer is 224, which demonstrates same padding preserving output size. This directly caught a factual error that would have been visible to the lecturer during the live presentation.

Prompt 2

Prompt: *"I don't really like the description of the Laplacian sharpening kernel, can you suggest a revision?"*

Response summary: Claude offered three alternative descriptions at different levels of technicality. The slope/peak-of-slope analogy was suggested as the strongest option because it directly contrasted Laplacian with Sobel, which was already introduced on the same slide, giving students a mental model for when to choose one over the other rather than just describing them in isolation.

Prompt 3

Prompt: *"Explain standard vs depthwise separable convolutions"*

Response summary: Claude broke down the two-step process clearly: depthwise filtering per channel followed by pointwise mixing, and explained why splitting the operations works: spatial patterns and channel relationships do not need to be learned together.

Prompt 4

Prompt: *"Zero-padding can introduce artificial edges at borders. Feature maps know they are near the edge, which distorts spatial representations. Make this better."*

Response summary: Claude rewrote the description explaining why zero-padding introduces a systematic bias. Kernels near the border multiply weights against fake zeros, producing weaker activations, and after many layers the network implicitly learns its position in the image rather than what is in the image. The condensed version of this was used directly on the failure cases slide.

Section 3: Five Questions

1. Which concept did you find hardest to explain, and why?

Depthwise separable convolutions were the hardest to explain. The concept splits into two steps: depthwise filtering per channel, then pointwise mixing across channels. Depthwise separable convolutions were the hardest to explain. The concept splits into two steps but it was not immediately clear why - if a standard convolution does both things at once, why does separating them still work? Understanding that spatial patterns and channel relationships are actually independent of each other was the key insight, but explaining that was hard.

2. Where did AI give you something that looked correct but turned out to be wrong?

When drafting the dimension equation slide, Claude generated a worked example: Input=224, K=3, P=1, S=1 → O = (224−3+0)/1 + 1 = 222. This looked correct at first glance, but there were two errors: the padding term was omitted from the calculation (written as 0 instead of 2×1=2), and the result was therefore wrong. The correct answer is 224. I caught this by manually working through the formula myself before finalising the slide, which confirmed the value of always verifying AI-generated calculations independently.

3. Describe a moment where using AI wasted your time.

I spent time prompting Claude for a good analogy for depthwise separable convolutions, going through a photography darkroom analogy and a paintbrush analogy before deciding neither added clarity over a direct technical explanation. The back-and-forth took longer than simply writing a clean explanation from scratch would have. I learned that for technical content where the mechanics are precise, AI-generated analogies often introduce more confusion, and a direct explanation with worked numbers is more useful.

4. Describe a moment where AI genuinely made your team faster.

The failure cases slide benefited significantly from Claude's descriptions. Concepts like checkerboard artifacts from transposed convolutions and the texture bias of CNNs are welldocumented but hard to describe accessibly in a few lines. Claude produced clear, technically accurate descriptions quickly and explained the underlying mechanism in each case, not just naming the failure. This saved substantial time compared to researching and writing each failure case from academic sources, and the quality was high enough that only minor edits were needed.

5. What question does your team still not feel confident answering?

After building the entire learning pack, the one question I still cannot answer confidently is why certain dilation rate sequences avoid blind spots in the receptive field while others do not. I know the problem exists and that d=1, 2, 4 is a common fix, but the reasoning behind why that sequence works, and how you would choose the right one for a different architecture, is something none of us felt we could explain clearly under pressure.

AI Usage Journal

ARI 2129 — Principles of Computer Vision for AI

Topic: Convolution Kernels • Author: Greg • Role: Simulation

Section 1: Tools Declared

The following AI tools were used during the production of the presentation slides deliverable.

Tool	Version	Purpose
Claude (Anthropic)	Claude Sonnet 4.6	Building the simulator UI, implementing kernel logic, debugging Python and Streamlit errors, and setting up the development environment.

Section 2: Notable Prompts

The following prompts represent moments that genuinely changed or improved the work, not routine interactions.

Prompt 1

Prompt: *" Build a Streamlit app skeleton with a two-column layout, controls on the left, output on the right."*

Response summary: Claude produced the full page config, column layout, and import structure in one response. This gave the simulator a clean, professional foundation immediately and meant no time was spent on boilerplate, allowing focus to go straight to implementing the kernel logic.

Prompt 2

Prompt: *" Add image upload and sample image selection to the simulator."*

Response summary: Claude produced a radio button toggle between upload and sample modes, with three built-in sample images (Checkerboard, Gradient, Noise) generated programmatically using numpy. This also introduced the RGB/BGR colour conversion pattern that was used throughout the rest of the app.

Prompt 3

Prompt: *" The Sobel output looks grey with white and black dashes rather than bright edges on black. Why and how do I fix it?"*

Response summary: Claude explained that normalised Sobel output maps zero to grey (128) rather than black, preserving sign information. Rather than simply fixing it, Claude suggested adding a toggle between absolute and normalised display modes so users can see both representations and understand what they mean. This became one of the most educational features of the simulator.

Prompt 4

Prompt: *" There are multiple slider elements with the same auto-generated ID.."*

Response summary: Claude identified that Streamlit generates element IDs from parameter names, so three kernels all having sliders named "Padding" and "Stride" caused a conflict. The fix was adding unique key arguments to each slider. Claude explained the root cause clearly rather than just patching it, which helped understand the error rather than just resolve it.

Section 3: Five Questions

1. Which concept did you find hardest to explain, and why?

Stride was the hardest concept to explain. Initially I did not fully understand it myself, it seemed like it was simply blurring the image. I eventually understood that stride does not blur at all; it controls how far the kernel jumps between positions, producing a smaller output image rather than a softer one. The confusion came from conflating the visual effect of a lower resolution output with the blurring effect of a Gaussian kernel, which look similar at a glance but are completely different operations.

2. Where did AI give you something that looked correct but turned out to be wrong?

When implementing the Sobel kernel, Claude produced code that normalised the output to the 0–255 range, which looked correct and ran without errors. However when tested on the checkerboard image the output appeared grey with alternating white and black dashes rather than clear bright edges. The normalisation was technically correct, it preserved the sign of the gradient, showing direction of change, but it was not what I wanted visually. I resolved this by adding a toggle between normalised and absolute display modes, so both representations are available to the user. This turned a limitation into a feature.

3. Describe a moment where using AI wasted your time.

When searching for key papers for the bibliography, the team attempted to use Claude to find relevant academic sources. Claude generated plausible-sounding citations that turned out to be hallucinated — the papers either did not exist or did not match the details provided. We spent time trying to locate these sources before realising they were fabricated. We learned that AI should not be trusted for source discovery and that academic papers need to be found and verified manually, either through Google Scholar or institutional databases.

4. Describe a moment where AI genuinely made your team faster.

Building the simulator UI was dramatically faster with AI assistance. The Streamlit layout, file uploader, radio buttons, sliders, and side-by-side image display were all produced quickly and cleanly, with no time spent on boilerplate. This freed up time to focus on the actual computer vision logic, the kernel implementations, the stride and padding controls, and the output dimension formula. The resulting UI was also cleaner and more polished than it would have been if built from scratch without assistance.

5. What question does your team still not feel confident answering?

The deep mathematics behind kernels remains difficult to explain confidently even after building the entire learning pack. While we can describe what each kernel does and apply the output dimension formula correctly, questions about the underlying calculus, such as why the Gaussian

formula takes the form it does, or how backpropagation adjusts kernel weights during training, are still hard to answer under pressure. This kind of mathematical intuition requires sustained practice and hands-on experience rather than just reading and building, and is something that cannot be fully developed in the timeframe of one assignment.

AI Usage Journal

ARI 2129 — Principles of Computer Vision for AI

Topic: Convolution Kernels • Author: Dimitrios • Role: Quiz and Rationale

---

Section 1: Tools Declared

The following AI tools were used during the production of the quiz and rationale.

Tool	Version	Purpose
ChatGPT	GPT-5.5	Assisting with convolution theory explanations, helping structure the cheat sheet, debugging convolution formulas, and refining educational content for the simulator and presentation materials.

Section 2: Notable Prompts

The following prompts represent moments that genuinely changed or improved the work, not routine interactions.

Prompt 1 Prompt:

“Explain convolution output dimensions clearly and give examples using stride and padding values.”

Response summary:

AI generated the standard convolution output formula and explained how kernel size, padding, and stride affect the final image dimensions. The explanation included worked numerical examples that

were later adapted directly into the Google Forms quiz and quick-reference cheat sheet. This helped the team simplify a mathematical concept into something easier for beginners to understand.

Prompt 2 Prompt:

"Create multiple-choice quiz questions about Gaussian, Sobel, and Laplacian filters, including common misconceptions as distractors." **Response summary:**

AI produced several quiz questions with intentionally misleading distractors based on typical student misunderstandings, such as confusing Gaussian blur with edge detection or misunderstanding Laplacian filters. This significantly improved the quality of the adversarial quiz because the questions tested understanding rather than memorisation.

Prompt 3

Prompt:
"Summarise the Xception paper and explain the difference between regular convolution and depthwise separable convolution in simple language."

Response summary:
AI converted dense academic language from the Xception paper into a simplified explanation suitable for the cheat sheet. It clarified how depthwise separable convolutions split spatial and channel processing, which helped the team connect theoretical CNN concepts to practical convolution operations used in area processing.

Prompt 4 Prompt:

"Generate feedback explanations for correct and incorrect quiz answers in Google Forms."

Response summary:

AI generated concise feedback for every answer choice, explaining both why the correct answer was right and why incorrect answers were plausible mistakes. This improved the educational value of the quiz because students could immediately understand their misconceptions instead of only seeing whether they were correct or incorrect.

Prompt 5 Prompt:

"Rewrite convolution formulas in a cleaner one-line format suitable for a cheat sheet."

Response Summary:
AI reformatted long mathematical expressions into compact single-line formulas that were easier to read and paste into documents and Google Forms. This improved the visual clarity of the

quickreference sheet and reduced formatting problems when transferring content between Word and Google Forms.

Section 3: Five Questions

1. Which concept in your topic did you find hardest to explain to someone else, and why do you think that was?

The hardest concept to explain was the relationship between stride, padding, and output dimensions in convolution operations. Initially, it was easy to memorise the formula but difficult to explain why changing stride reduces output size or why padding preserves spatial dimensions. The challenge came from the fact that these parameters interact together mathematically and visually at the same time. Understanding improved only after testing several worked examples and seeing how kernels move across images step by step.

---

2. Where did AI give you something that looked correct but turned out to be wrong or incomplete? What did you do about it?

When generating quiz questions, AI initially produced some technically correct answers but paired them with weak distractors that were too obviously incorrect. This would not properly test deep understanding. In another case, AI oversimplified the explanation of depthwise separable convolutions and omitted the role of pointwise convolutions entirely. To fix this, the team manually reviewed the questions, checked explanations against the Xception paper, and rewrote several distractors to better reflect realistic misconceptions students might have.

---

3. Describe a specific moment where using AI wasted your time. What happened and what did you learn from it?

A major waste of time happened while trying to use AI to locate academic references related to convolution filters and CNN architectures. AI generated several realistic-looking citations and paper summaries that either did not exist or had incorrect publication details. Time was spent searching for these sources before realising they were hallucinated. This taught the team that AI is useful for summarising verified papers but unreliable for academic source discovery unless every citation is independently checked through trusted databases like Google Scholar.

4. Describe a specific moment where AI genuinely made your team faster or produced something better than you would have managed alone. What made the difference?

AI dramatically accelerated the creation of the adversarial quiz and quick-reference cheat sheet. Instead of manually brainstorming misconceptions, AI quickly generated multiple plausible distractors and concise explanations for each answer. It also simplified dense academic material from the Xception paper into beginner-friendly summaries. The difference was speed and iteration — the team could rapidly test different explanations and refine the educational material rather than spending hours drafting everything manually from scratch.

---

5. What question about your topic does your team still not feel confident answering, even after building the entire Learning Pack?

The team still does not feel fully confident explaining the deeper mathematical reasoning behind why certain kernels work so effectively, especially Gaussian and Laplacian filters. While it is possible to describe their practical purpose, questions involving the calculus, frequency-domain interpretation, or why specific kernel weights are mathematically optimal remain difficult.

AI Usage Journal

ARI 2129 — Principles of Computer Vision for AI

Topic: Convolution Kernels • Author: Nathan • Role: Jupyter Notebook

Section 1: Tools Declared

The following AI tools were used during the production of the Jupyter notebook deliverable.

Tool	Version	Purpose
Claude (Anthropic)	Claude Sonnet 4.6	Explaining concepts during notebook development, checking code logic, clarifying parameter behaviour, and answering questions about kernel maths.

Section 2: Notable Prompts

The following prompts represent moments that genuinely changed or improved the work, not routine interactions.

Prompt 1

Prompt: *"Why do we convert the image to float before doing anything to it?"*

Response summary: Claude explained that uint8 values overflow on arithmetic — 250 + 10 wraps to 4 rather than 260. Converting to float in [0, 1] first means filtering operations produce correct results. This clarified why img_as_float is the first step rather than an optional one.

Prompt 2

Prompt: *"Why does increasing sigma blur more, what is it actually controlling?"*

Response summary: Claude explained that sigma controls the width of the Gaussian bell curve, so a larger sigma means the kernel assigns meaningful weight to pixels further from the centre, effectively averaging over a wider neighbourhood. This went into the parameter table explanation in section 3.

Prompt 3

Prompt: *"Why do we need both Gx and Gy, why not just one Sobel kernel?"*

Response summary: Claude explained that Gx only responds to edges running vertically and Gy only to horizontal ones. A diagonal edge would be partially missed by either alone. Combining them with sqrt(Gx^2 + Gy^2) gives a response to edges at any angle. This shaped the explanation written under the Sobel section.

Prompt 4

Prompt: *"How do I detect zero-crossings in the LoG output without looping over every pixel?"*

Response summary: Claude suggested using scipy.ndimage.generic_filter with a function that checks whether the centre pixel times any neighbour is negative, since opposite signs always

give a negative product. This replaced what would have been a slow nested loop and is the approach used in section 6.3.

Prompt 5

Prompt: *"Why does sharpening subtract the Laplacian instead of adding it?"*

Response summary: Claude explained that with the -8 centre convention, the Laplacian is negative at bright peaks. Subtracting a negative value adds brightness there, which is the sharpening effect. This clarified the sign logic used in the unsharp masking cell and the comment explaining it.

Prompt 6

Prompt: *"How do I force the kernel size to always be odd without an if-check?"*

Response summary: Claude suggested the bitwise trick int(6 * sigma + 1) | 1, which sets the lowest bit so any even result becomes the next odd number up and any already-odd number stays the same. Used in every kernel size calculation throughout the notebook.

Section 3: Five Questions

1. Which concept did you find hardest to explain, and why?

The Laplacian was the hardest to explain. Gaussian and Sobel both have an intuitive description — one blurs, one finds edges. The Laplacian required explaining second derivatives, why zero-crossings mark edge locations more precisely than gradient magnitude does, and why the sign of the kernel means sharpening requires subtraction rather than addition. Each of those ideas depends on the previous one so the explanation had to be built up carefully rather than stated directly.

2. Where did AI give you something that looked correct but turned out to be wrong?

When asking about the kernel size rule, Claude initially described k = 6*sigma as sufficient without mentioning the +1 offset needed to ensure the size covers the full bell curve. The formula ran without errors but produced even kernel sizes at certain sigma values, which caused an assertion error. I had to go back and clarify, and Claude then gave the correct rule with the |1 trick. It was a small omission but showed that the first answer is not always complete.

3. Describe a moment where using AI wasted your time.

I spent time asking Claude for the best way to implement convolution from scratch to include in the notebook, going back and forth on whether to use a manual loop or a vectorised version. After several exchanges I decided not to include a from-scratch implementation at all because the formula in the markdown already covers the concept and the code cells are cleaner using scipy. The time spent debating the implementation would have been better spent just writing the explanation.

4. Describe a moment where AI genuinely made you faster.

The zero-crossing detection in section 6.3 would have taken significant time to figure out independently. I knew what I wanted to show but had no clean way to express 'sign flip with at least one neighbour' without a loop. Claude gave the generic_filter approach with the product-sign condition immediately and explained why it works. That cell went from a problem to a working visual in a few minutes.

5. What do you still not feel fully confident answering?

Why the LoG at different sigma values picks up features at different scales but Sobel at the same sigmas does not show the same scale selectivity as cleanly. I understand that sigma controls the spatial extent of the filter and larger sigma suppresses fine detail, but being able to explain precisely why zero-crossings at large sigma correspond to coarser structures rather than just saying 'it blurs more first' is something I could not fully articulate under pressure.